A

MINI PROJECT REPORT

ON

FARMER'S LINKDLN

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

G.HARINADH BABU (23R91A05A7)

Under the Guidance

Of

Mrs. A.MOUNIKA

Assistant Professor

Department of CSE



Department of Computer Science and Engineering

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(An Autonomous Institution)

Medbowli, Meerpet, Saroornagar, Hyderabad – 500097

(Affiliated to JNTUH, Approved by AICTE, Accredited by NBA & NAAC)

(2023-2027)

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)
Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.
Phone: 040-24092838 Fax: +91-040-24092555
E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in
Department of Computer Science & Engineering

College code: R9

CERTIFICATE

This is to certify that the Mini Project report on **"FARMER'S LINKDLN"** is a bonafide work carried out by D. Chaitanya Varma (23R91A0569), G. Harinadh Babu (23R91A05A7), I. Rishi Adwith Swastik (23R91A05B8) in partial fulfillment for the requirement of the award of B.Tech degree in Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Hyderabad, affiliated to Jawaharlal Nehru Technological University, Hyderabad under my guidance and supervision.

The result of investigation enclosed in this report have been verified and found satisfactory. The results embodied in the project work have not been submitted to any other University for the award of any degree.

INTERNAL GUIDE

Mrs. A. Mounika

Assistant Professor

……..

HEAD OF DEPARTMENT

Dr. CH. V. Phani Krishna

Professor

……..

EXTERNAL EXAMINER

……..……

PRINCIPAL

Dr. K. Venkata Murali Mohan

Professor

……

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)
Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.
Phone: 040-24092838 Fax: +91-040-24092555
E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in
Department of Computer Science & Engineering

College code: R9

DECLARATION

We hereby declare that the Mini Project report entitled **"FARMER'S LINKDLN"** is done under the guidance of **Mrs. A. Mounika (Assistant Professor),** Department of Computer Science and Engineering, Teegala Krishna Reddy Engineering College, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Jawaharlal Nehru Technological University**, Hyderabad.

This is a record of bonafide work carried out by us in **Teegala Krishna Reddy Engineering College** and the results embodied in this project have not been reproduced or copied from any source.

Submitted by

G.HARINADH BABU **(23R91A05A7)**

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowned our efforts with success.

We extend my deep sense of gratitude to **Dr. K. Venkata Murali Mohan**, **Principal,** Teegala Krishna Reddy Engineering College, Meerpet, for permitting me to undertake this project.

We are indebted to **Dr.CH.V.Phani Krishna, Professor & Head of the Department**, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet for his support and guidance throughout our project.

We are indebted to our guide **Mrs. A.Mounika**, Assistant Professor, Computer Science Engineering, Teegala Krishna Reddy Engineering College, Meerpet for his/her support and guidance throughout our project.

We are indebted to the project coordinator **Mrs. P. Swetha,** Assistant Professor, and **Mrs. A. Mounika**, Assistant Professor, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet for their support and guidance throughout our project.

Finally, we express thanks to one and all that have helped me in successfully completing this mini project. Further I would like to thank my family and friends for their moral support and encouragement.

By

G.HARINADH BABU (23R91A05A7)

ABSTRACT

This project presents a smart digital platform that connects farmers, agricultural workers, and landowners on a single network, much like a professional linking system for agriculture. The platform aims to close the gap between labor demand and availability with a centralized and easy- to-use system. It includes AI-based worker recommendations that consider factors like location, skills, availability, ratings, and cost to ensure efficient matching. A wage prediction module is part of the platform to support fair pricing for both farmers and workers. The system also allows for crop and season-based labor planning, helping farmers estimate their workforce needs effectively. To enhance accessibility for rural users, it has voice-based interaction with Telugu language support. Additional features like worker availability calendars, emergency labor requests, and land- sharing options increase flexibility and usability. The platform also provides real-time weather alerts to help farmers make informed decisions. Overall, it improves efficiency, transparency, and trust in agricultural labor and land management while using technology to address real challenges in the agricultural sector.

LIST OF FIGURES

Figure No.	Figure Description	Page No.
4.1	System Architecture	06
4.2	Class Diagram	07
4.3	Use Case Diagram	08
4.5	Activity Diagram	09
4.5	Sequence Diagram	10
8.1	Login Page	21
8.2	Register Page	21
8.3	User Page	22
8.4	Land Posting Page	22
8.5	Workers Hiring Page	23
8.6	Jobs Posting Portal	23

CONTENTS

	TITLE	PAGE NO
1.	**INTRODUCTION**	
	1.1 PROBLEM STATEMENT	01
	1.2 DESCRIPTION	01
2.	**LITERATURE SURVEY**	02
3.	**SYSTEM ANALYSIS**	
	3.1 EXISTING SYSTEM	03
	3.2 PROPOSED SYSTEM	03
	3.3 SYSTEM REQUIREMENTS	04
4.	**SYSTEM DESIGN**	
	4.1 SYSTEM ARCHITECTURE	05
	4.2 UNIFIED MODELING LANGUAGES (UML) DIAGRAMS	06
5.	**IMPLEMENTATION**	
	5.1 MODULES	11
	5.2 DEVELOPMENT TOOLS USED	12
	5.3 TECHNOLOGIES USED	13
6.	**CODING**	14
7.	**SYSTEM TEST**	
	7.1 BASICS OF SOFTWARE TESTING	19
	7.2 Testing Approach	19
	7.3 Functional Testing	19
	7.4 Non-Functional Testing	20
	7.5 User Acceptance Testing	20
8.	**OUTPUT SCREENS**	21
9.	**CONCLUSION**	24
10.	**FUTURE ENHANCEMENT**	25
11.	**REFERENCES**	27

CHAPTER-1
INTRODUCTION

1.1 Problem Statement

The Indian economy is largely agricultural but the farmers have been struggling with labour and land resource management. The existing hiring operation is not well structured and depends on middlemen, personal contacts and word of mouth, which creates delays and inefficiencies, particularly during high seasons. Employees do not have a systematic job avenue and farmers are unable to access skilled and available labour in a timely manner. Current agricultural uses are mostly in the areas of advisory services and market prices with little regard to the needs of labour management and land-sharing

1.2 Description

Agriculture is still the Indian economy that supports the livelihoods of a good number of the population. Although it is important, there are still a number of operational issues that farmers encounter in their daily activities especially in terms of finding quality agricultural labour, setting out the fair rates of wages and making optimal use of land resources available. Personal contacts and word-of-mouth communication continue to be informal forms of labour management in most rural areas. This has been known to cause inefficiency such as wasting time in searching labour force, wage disputes and lack of labour in seasons of high agricultural production.

Despite the presence of a number of digital solutions in the agricultural field, the majority of these solutions are devoted to advisory services, equipment rental, or price monitoring on the market. The critical element of labour management and land utilization is discussed in a very limited number of platforms in a structured and scalable form. Consequently, farmers still stick to the traditional ways that are time-consuming and not transparent.

The proposed project presents AgroConnect, an intelligent agricultural human resource and land management system that is meant to overcome these problems. The system will include AI-based worker suggestions, availability tracking, and a voice interface with regional language support as an implementation to make the system more accessible to rural users. AgroConnect will help enhance productivity in the agricultural sector by giving a centralized and easy to use platform through which efficiency, transparency, and trust in agricultural labour management and land sharing will be increased.

CHAPTER-2
LITERATURE SURVEY

Over the past years, agriculture has experienced a great technological change, and many digital platforms have emerged to assist farmers in different fields like crop advisory, market price analysis, and equipment rental. Such applications as Kisan Sarathi, Farmitra, KhetEase, and FARMS - Farm Machinery Solutions can be very useful as they can offer useful information connected with weather conditions, crop management, government schemes, and access to farm machineries. Such systems have enhanced awareness and decision-making in the farmers particularly in rural areas.

Nevertheless, the majority of these solutions that are currently in place are based on advisory and informational support instead of the essential problem of labor management in agriculture. Hiring labor is still a heavily traditional process, which mostly relies on personal contacts, local labor contractors and word-of-mouth communication. The absence of a formal computer-based system results in inefficiencies like time wastage in locating employees, absence of transparency in wage payment, and inability to control labor supply, especially during harvesting seasons.

The recent advancements in the fields of artificial intelligence and mobile technology have provided chances to develop smarter and more efficient agricultural systems. Intelligence-based suggestions, real-time availability monitoring, voice are all features that can greatly improve usability and accessibility among rural customers. Nevertheless, no unified platform which integrates labor management, wage transparency and land-sharing within a cohesive system has been created yet.

AgroConnect system will be suggested to fill this gap as it will offer a single digital platform that will link farmers, workers, and landowners. It combines the latest technologies, including AI-based worker recommendation, availability tracking, and voice support to enhance efficiency, transparency, and trust in agriculture activities. AgroConnect will provide a complete solution to agricultural labor and land management by solving the shortcomings of current systems.

CHAPTER-3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing system, agricultural labor management is mostly carried out through manual methods such as personal contacts, local labor contractors, and word-of-mouth communication. While several agricultural mobile applications are available, they primarily focus on advisory services, farm machinery rental, and market price information rather than labor hiring and land management. Applications like FARMS – Farm Machinery Solutions, Kisan Sarathi, Farmitra, and KhetEase support farmers with guidance and information but do not provide a structured digital system for workforce availability, wage transparency, or land-sharing. As a result, farmers face operational inefficiencies, especially during peak agricultural seasons.

Key Features of the Existing System:

- Finding workers takes more time
- No clear idea of worker availability
- Frequent misunderstanding and disputes
- Not useful during urgent labour needs

3.2 PROPOSED SYSTEM

The suggested system is an intelligent digital platform that links farmers, agricultural workers and landowners using one application.The proposed system is a smart digital platform that connects farmers, agricultural workers, and landowners through a single application. It applies AI-based suggestions to propose the appropriate workers according to the location, skills, availability, and ratings. It also includes the system estimation of wages, voice interaction, and real-time tracking of the availability of workers. There are other features like emergency labour requests, land-sharing options, and weather alerts that aid farmers in making better decisions. Generally, the platform eases the process of hiring labour and managing land and enhances transparency and efficiency.

Key Features and Advantages:

- Easy and quick worker selection
- Voice support for rural users

- Real-time worker availability tracking
- Emergency labour support
- Better use of land resources

3.3 SYSTEM REQUIREMENTS

3.3.1 SOFTWARE REQUIREMENTS

- Coding Language : Java
- Framework : Spring Boot
- Database : MySQL
- Operating System: Windows
- Front-End : HTML, CSS, JavaScript
- IDE / Tools : Maven
- Build Tool : Postman / Swagger (Browser)

3.3.2 HARDWARE REQUIREMENTS

- System : Intel Core i5 or above
- Hard Disk : 80 GB (minimum)
- RAM : 8 GB (minimum)
- Processor Speed : 2.0 GHz or higher

CHAPTER-4

SYSTEM DESIGN

The system design of AgroConnect platform incorporates a combination of various modules in a central web-based application that is simplifying agricultural labor management and land sharing. The system starts by enabling users (farmers, workers and landowners) to create accounts and log-in with a user-friendly system. After authentication, farmers will be able to place job requirements, land availability, and workers, as well as kavulu farmers, will be able to search and apply.

All the business logic such as; job posting, worker applications, direct hiring and land requests are processed by backend which is written in Java Spring Boot. Its single workflow is that all the job applications and direct hiring are handled using the same application module and so the system is consistent and not complex. The information is saved in MySQL database all the records concerning users, jobs, applications, and land requests are organized in a systematic way.

4.1 SYSTEM ARCHITECTURE

AgroConnect system architecture is a layered and modular structure which guarantees free flow of data among components. On the initial level, the system receives user input via special web-based interfaces, such as the Farmer, Worker, Kavulu Farmer, and Admin Dashboards, where the user can control jobs, request work, or manage land-sharing requests.

This input is sent to the Application Layer which runs on a Spring Boot Application Server. This layer receives all user requests through HTTP Requests and handles fundamental modules such as Authentication, Job Management, Worker Application, Land Sharing and Notifications.

This backend is designed with a special Worker Availability Engine and Privacy Control logic to make sure that contact information is disclosed only upon the selection, and the workflow is secure and professional. The backend connects to the Database Layer via CRUD Queries in which a MySQL Database is utilized to store all systemic data, such as user profiles, job postings, and land records. It is interesting to note that the Admin Dashboard has Direct Admin Data Access, which provides high speed and 90 degree bypass connection to the MySQL Database to monitor and oversee real time.

All this is hosted on Local System Infrastructure, bringing about scalable and self-sustained environment. This modular architecture is a guarantee that allows effective communication among system components, which can offer an efficient and responsive experience to all stakeholders in the agricultural sector.

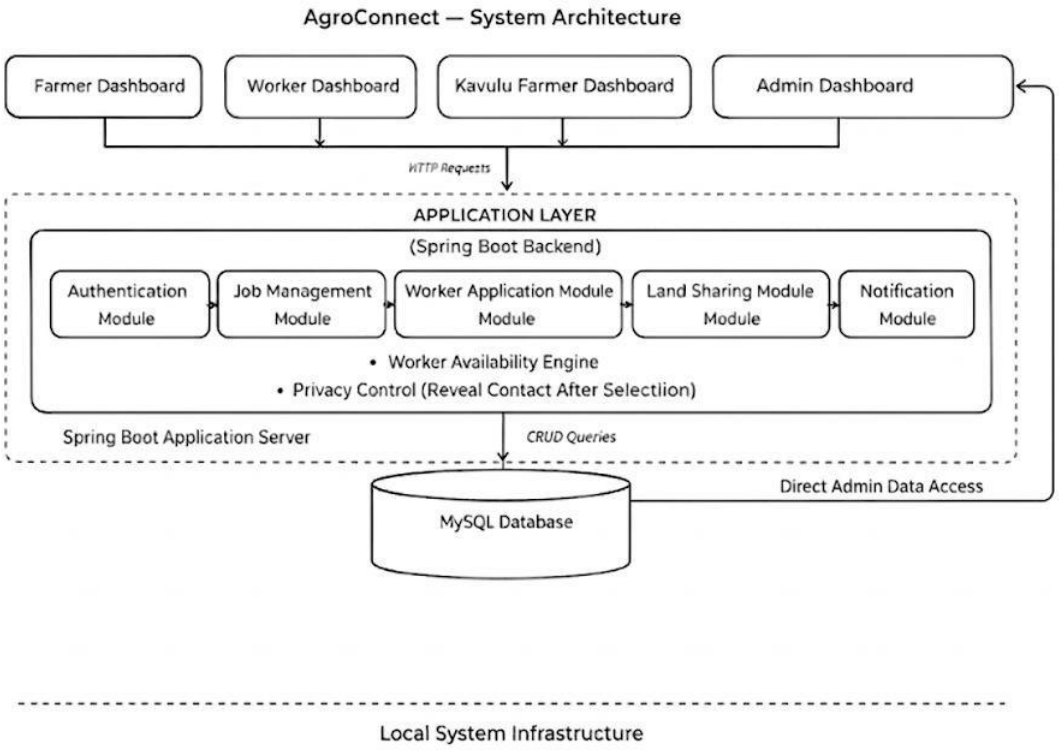


Fig. 4.1 System Architecture

4.2 Unified Model language(UML) Diagrams

Unified Modeling Language is a graphical language used for visualizing, constructing, and documenting the artifacts. UML Diagrams play a crucial role insoftware development and used graphical notations to express software project designs.

The UML Diagrams used in this paper are:

i. Class Diagram
ii. Use Case Diagram
iii. Activity Diagram
iv. Sequence Diagram

i. Class Diagram

The class diagram of the AgroConnect system represents the object-oriented structure of the application. The core classes include User, Job, Application, Land, and LandRequest. The User class stores details such as name, role (farmer, worker, kavulu), contact information, and login credentials. The Job class contains information related to job postings such as job details, location, duration, and farmer association.

The Application class acts as a central component, linking workers and jobs, and includes attributes such as application status (pending, selected, rejected) and type (applied or direct). The Land and LandRequest classes manage land-sharing functionalities, allowing kavulu farmers to request land and farmers to approve or reject those requests. These classes interact with each other through defined relationships, forming a modular and scalable system design.

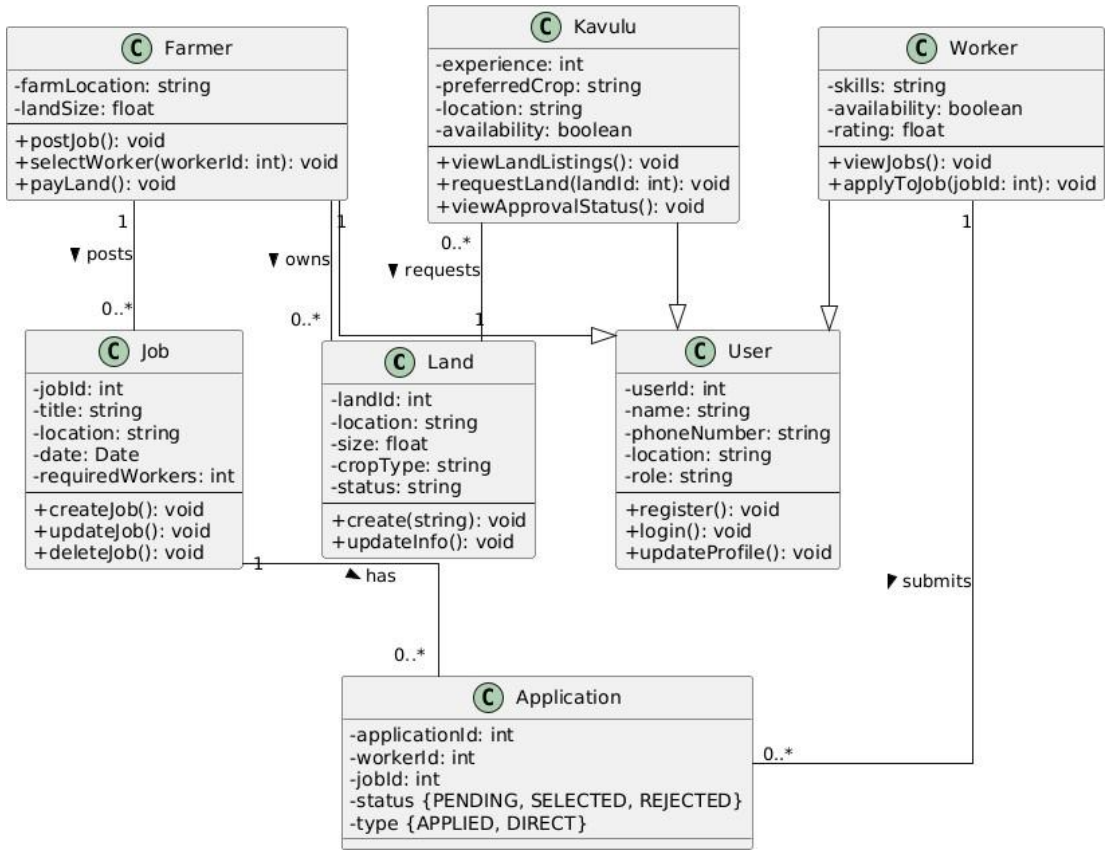


Fig. 4.2 Class Diagram

ii. Use Case Diagram

The use case diagram illustrates the interaction between users and the AgroConnect system. The primary actors in the system are Farmer, Worker, and Kavulu Farmer. Farmers can perform actions such as registering, logging in, posting jobs, viewing applications, selecting workers, and posting land. Workers can register, log in, search for jobs, apply for jobs, and update their availability. Kavulu farmers can request land and track approval status.

The diagram highlights how each user interacts with different functionalities of the system and provides a clear overview of system capabilities from the user's perspective.

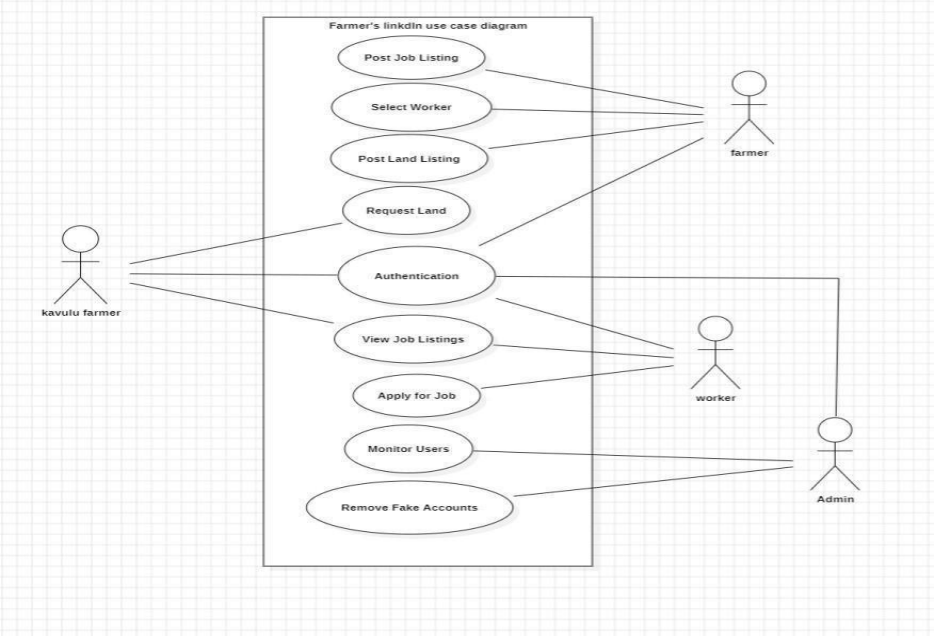


Fig. 4.3 Use Case Diagram

iii. Activity Diagram

The activity diagram represents the workflow of the AgroConnect system. It starts with user registration and login, followed by role-based actions. Farmers can either post jobs or search for workers. If a job is posted, workers can apply, and the farmer can review and select applicants. Alternatively, farmers can directly hire workers based on availability.For land management, farmers post land details, and kavulu farmers can request land, which is then approved or rejected by the farmer. The diagram shows decision points, parallel processes, and the overall flow of activities within the system

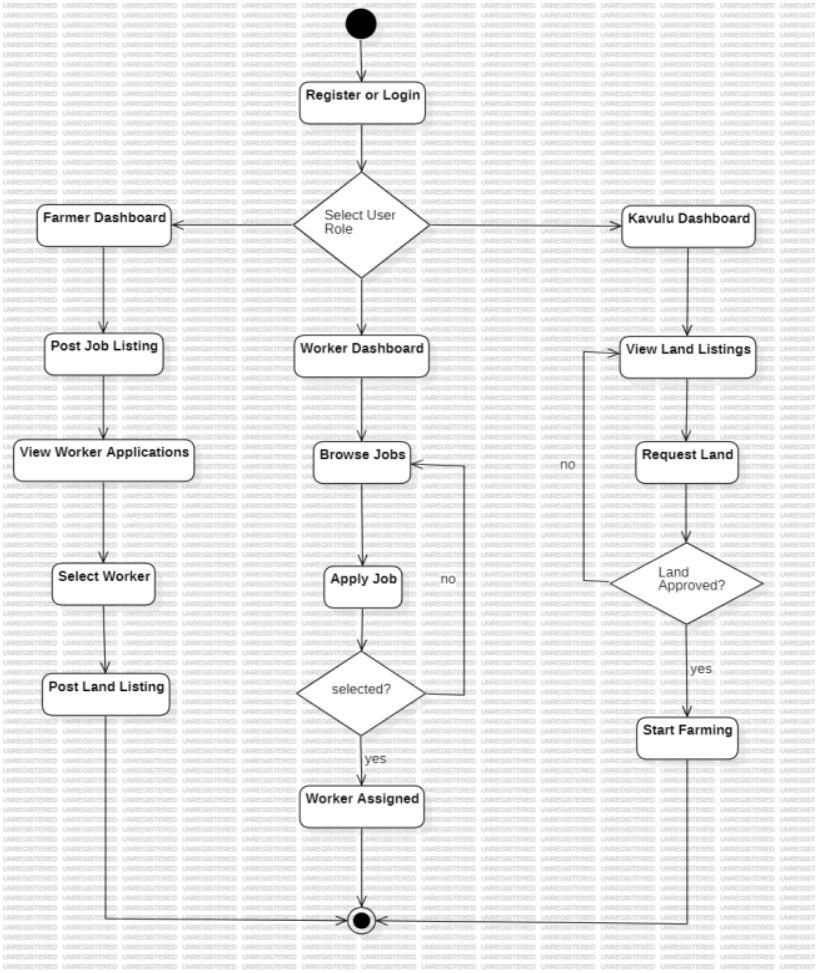


Fig. 4.4 Activity Diagram

iv. Sequence Diagram

The sequence diagram is used to show how the User, Frontend, Backend, and Database modules of AgroConnect system interact in the process of job application and worker selection. It depicts the chronological sequence of messages, which begins with the user posting a job application, continues with the front and backend request processing, and concludes with the storage of the data in the database. The diagram also illustrates that the farmer makes a selection of a worker which causes updates in the application status and verifies the selection. This order provides a suitable coordination and data integrity among all the components of the system.

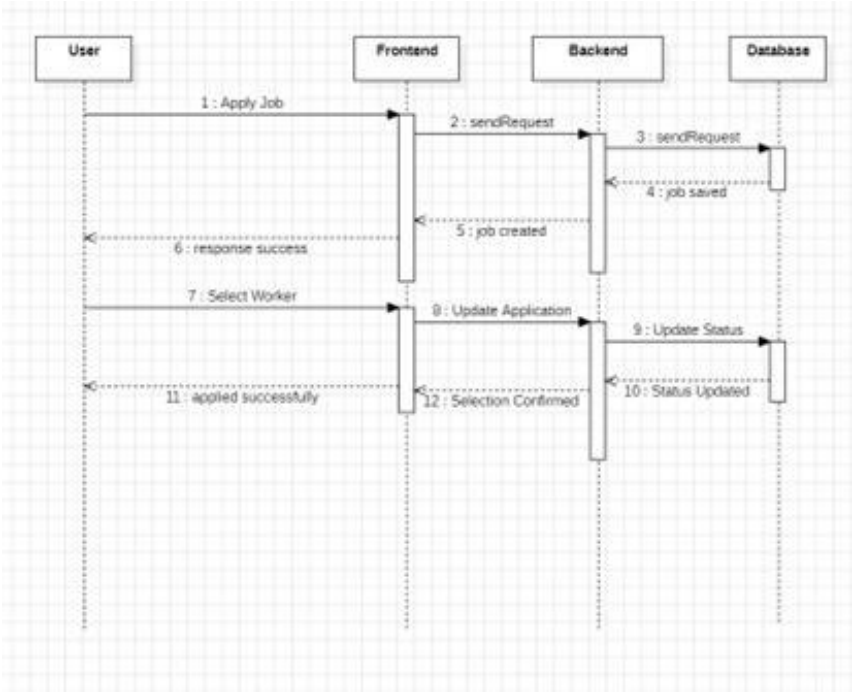


Figure. 4.5 Sequence Diagram

CHAPTER-5
IMPLEMENTATION

5.1 Modules

In this project, AgroConnect is divided into multiple functional modules based on the system architecture of agricultural workforce and land management.

5.1.1 User Authentication Module

This module handles secure access, registration, and role-based identity management for all platform participants.

a. Users register using their name, a validated 10-digit mobile number, and location.

b. The system implements role-based classification (Farmer, Worker, Kavulu).

c. Secure login functionality is managed via JWT (JSON Web Tokens) for stateless authentication.

d. Mobile number validation and basic OTP logic ensure account authenticity.

e. User credentials and profile metadata are stored securely in the MySQL database.

5.1.2 Job Management Module

This module provides the interface for labor requirement coordination and job lifecycle management.

a. Farmers can create detailed job postings including title, duration, and worker requirements.

b. Real-time job data is categorized by location for easier discovery by workers.

c. The module supports dynamic updates, allowing farmers to modify or delete active listings.

d. Workers can browse a filtered list of available jobs based on their proximity and availability.

5.1.3 Land Management Module

This module facilitates the "Kavulu" ecosystem by managing land assets and sharing requests.

a. Farmers can list land parcels with specific attributes like size, location, and suitable crop types.

b. The system maintains an availability status for each land entry to prevent redundant requests.
c. Kavulu users can browse available listings and initiate land-sharing requests.
d. The module handles the approval workflow, updating request statuses dynamically from pending to approved or rejected.

5.1.4 Admin Management Module

This module serves as the central oversight authority for platform integrity and data monitoring.

a. Admins have a unified view of all registered users, active jobs, and pending applications.
b. The module includes data moderation tools to delete inappropriate or fraudulent listings
c. Platform activity monitoring ensures the system operates within defined agricultural parameters.
d. Admin-level dashboards provide insights into the total workforce and land utilization metrics.

5.2 Development Tools used

5.2.1 Java(Spring Boot)

Spring Boot Java is used to develop the backend. It offers a solid, enterprise-level application framework to create RESTful API and manage complex business logic with dependency injection and scalable design.

5.2.2 MySQL

MySQL is used as the primary relational database for storing structured data such as users, jobs, applications, and land details, ensuring high data consistency and efficient query handling.

5.2.3 Visual Studio Code / IntelliJ IDEA

These Integrated Development Environments (IDEs) are used for writing, debugging, and managing the Java Spring Boot source code, providing essential tools for rapid development.

5.2.4 Postman

Postman is utilized for testing and verifying the REST API endpoints. It helps in validating JSON data exchange between the frontend and the backend services.

5.3 Technologies Used

5.3.1 Spring Boot Framework

Spring Boot simplifies the development of the backend by providing pre-configured modules. Key features include REST API development, security integration, and seamless database connectivity via JPA/Hibernate.

5.3.2 REST API Architecture

The system employs a RESTful architecture for communication. This ensures stateless communication and efficient JSON-based data exchange between the user interface and the server.

5.3.3 AWS (Cloud Infrastructure)

The application utilizes AWS EC2 for hosting the backend server and AWS RDS for managing the MySQL database, ensuring high availability and scalability for the platform.

CHAPTER-6

CODING

User.java (Core Entity Model)

```
@Entity
@Table(name = "users")
public class User
{
    public enum Role { FARMER, WORKER, KAVULU, ADMIN }
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true, length = 10)
    private String phone;
    @Column(nullable = false)
    private String password;
    @Enumerated(EnumType.STRING)
    private Role role;
    private String village;
    private String district;
    private String workType;
    @Column(nullable = false)
    private Boolean verified = false;
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
```

```
    @PrePersist
    protected void onCreate() { this.createdAt = LocalDateTime.now(); }
}
```

Job.java (Labor Requirements Model)

```
@Entity
@Table(name = "jobs")
public class Job
 {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long jobId;
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "farmer_id", nullable = false)
    private User farmer;
    @Column(name = "work_type", nullable = false)
    private String workType;
    private Integer durationDays;
    private Integer workersRequired = 1;
    private String district;
    @PrePersist
    protected void onCreate() { this.createdAt = LocalDateTime.now(); }
}
```

Database Schema Implementation

```
CREATE TABLE IF NOT EXISTS users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    phone VARCHAR(10) NOT NULL UNIQUE,
    role ENUM('FARMER', 'WORKER', 'KAVULU', 'ADMIN') NOT NULL,
    verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS job_applications (
    application_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    job_id BIGINT NOT NULL,
    worker_id BIGINT NOT NULL,
    status ENUM('PENDING', 'SELECTED', 'REJECTED') DEFAULT 'PENDING',
    FOREIGN KEY (job_id) REFERENCES jobs(job_id) ON DELETE CASCADE,
    FOREIGN KEY (worker_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Farm Controller Implementation

```
package com.agroconnect.controller;
import com.agroconnect.service.FarmService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.*;
import java.util.Map
```

```
@RestController
@RequestMapping("/api/farm")
public class FarmController {
    private final FarmService farmService;
    public FarmController(FarmService farmService) {
        this.farmService = farmService;
    }
    @PostMapping("/jobs")
    public ResponseEntity<?> postJob(@RequestBody Map<String, String> body) {
        farmService.postJob(getCurrentUserId(), body);
        return ResponseEntity.ok(Map.of("message", "Job posted"));
    }
    @GetMapping("/jobs")
    public ResponseEntity<?> getAllJobs(@RequestParam(required = false) String district) {
        return ResponseEntity.ok(farmService.getAllJobs(district));
    }
    @PostMapping("/jobs/{jobId}/apply")
    public ResponseEntity<?> applyForJob(@PathVariable Long jobId) {
        farmService.applyForJob(jobId, getCurrentUserId());
        return ResponseEntity.ok(Map.of("message", "Applied"));
    }
    @PutMapping("/applications/{appId}/select")
    public ResponseEntity<?> selectWorker(@PathVariable Long appId) {
        farmService.selectWorker(appId, getCurrentUserId());
```

```
        return ResponseEntity.ok(Map.of("message", "Worker selected"));
    }
    @PostMapping("/land")
    public ResponseEntity<?> postLand(@RequestBody Map<String, String> body) {
        farmService.postLand(getCurrentUserId(), body);
        return ResponseEntity.ok(Map.of("message", "Land posted"));
    }
    @PostMapping("/land/{landId}/request")
    public ResponseEntity<?> requestLand(@PathVariable Long landId) {
        farmService.requestLand(landId, getCurrentUserId());
        return ResponseEntity.ok(Map.of("message", "Request submitted"));
    }
    @GetMapping("/workers")
    public ResponseEntity<?> getAvailableWorkers(@RequestParam(required = false) String district)
{
        return ResponseEntity.ok(farmService.getAvailableWorkers(district));
    }
    @PostMapping("/hire-worker/{workerId}")
    public ResponseEntity<?> hireWorker(@PathVariable Long workerId, @RequestBody
Map<String, Integer> body) {
        farmService.hireWorkerDirectly(getCurrentUserId(), workerId, body.get("durationDays"));
        return ResponseEntity.ok(Map.of("message", "Worker hired"));
    }
```

CHAPTER-7

SYSTEM TEST

7.1 Basics of Software Test

System testing is performed to evaluate the complete AgroConnect platform and ensure that all integrated modules function correctly as a single unit. This phase verifies that both functional requirements (Job posting, Land requests) and non-functional requirements (Security, Performance) meet the initial design specifications by simulating real-world agricultural coordination scenarios.

7.2 Testing Approach

The system was validated using a combination of Manual Testing and Automated API Testing.

- **Unit Testing**: Each Java Entity (User, Job) was tested for data integrity.
- **API Testing**: Postman was utilized to send REST requests to the Spring Boot controllers to verify JSON responses and HTTP status codes (e.g., 200 OK, 201 Created).
- **Integration Testing**: End-to-end flows, such as a Farmer posting a job and a Worker immediately seeing it in their feed, were verified to ensure seamless data flow between the backend and the MySQL database.

7.3 Functional Testing (Test Cases)

Functional testing ensures that every feature of the AgroConnect system operates according to the business logic. The results of the key test cases are summarized in the table below:

Test ID	Test Case Description	Expected Result	Status
Test-01	User Registration	New user account created in database	Passed
Test-02	User Login (JWT)	Valid credentials return a success token	Passed
Test-03	Post Job Requirement	Job details persisted with Farmer ID	Passed
Test-04	Job Application	Worker ID linked to Job ID in application table	Passed
Test-05	Worker Selection	Application status updated to 'SELECTED'	Passed

Test-06	District Filtering	Jobs filtered correctly based on location	Passed
Test-07	Land Posting (Kavulu)	Land area and crop type stored successfully	Passed
Test-08	Land Request	Kavulu request submitted to Farmer dashboard	Passed
Test-09	Role-Based Access	Workers restricted from Admin/Farmer routes	Passed

7.4 Non-Functional Testing

Non-functional testing evaluates the operational quality and robustness of the architecture.

- **Performance:** The system demonstrates stable response times for concurrent API calls, ensuring the Spring Boot application handles multiple worker searches without latency.
- **Security:** Implementation of Spring Security and JWT ensures that user data is protected and that only authenticated users can perform sensitive actions like hiring or land approval.
- **Usability:** The RESTful architecture provides a consistent data format (JSON), facilitating a smooth and intuitive user experience for frontend integration.
- **Reliability:** Database constraints and ON DELETE CASCADE logic ensure that the system maintains referential integrity even during data modifications.

7.5 User Acceptance Testing (UAT)

The system was evaluated from the end-user perspective to ensure it meets the practical needs of the agricultural community. During UAT, simulated users (Farmers and Workers) were able to complete the full lifecycle of hiring and land requesting without technical obstacles.

CHAPTER-8

OUTPUT SCREENS



Fig. 8.1 LOGIN PAGE

The given output screen is about the login page of the agroconnect web application

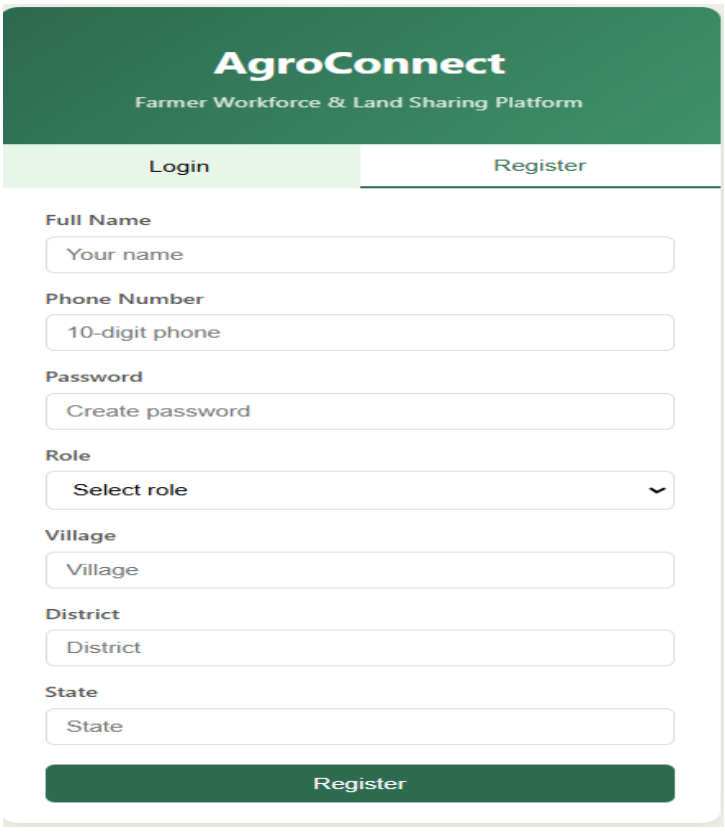


Fig. 8.2 REGISTER PAGE

The given output screen is about the registration page of the agroconnect web application

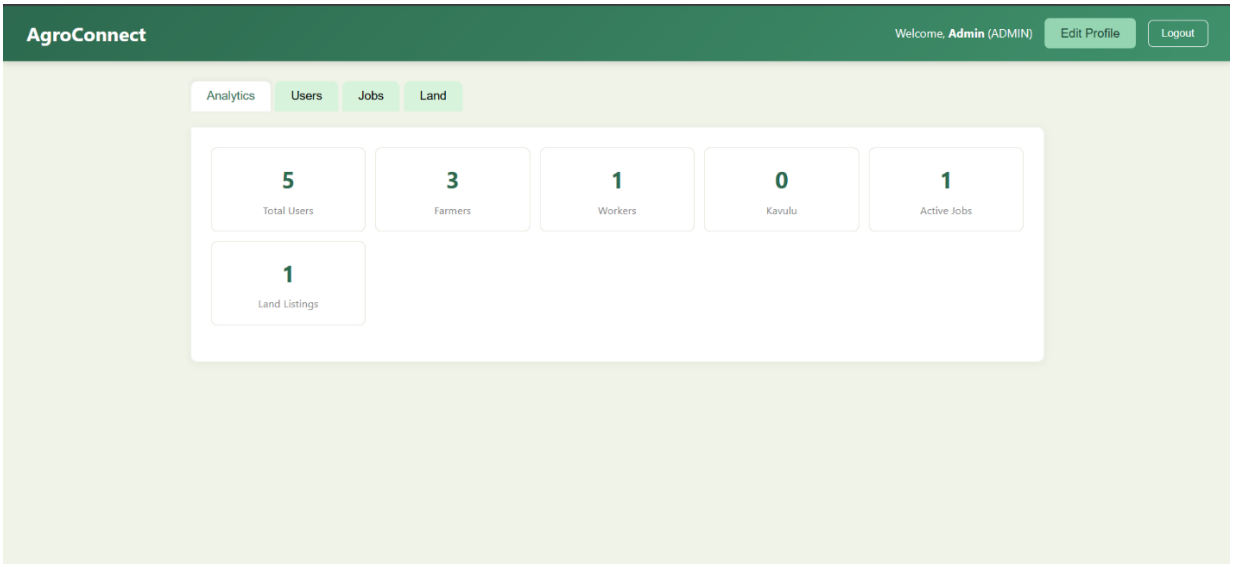


Fig. 8.3 USER PAGE

The above image describes about the admin page where the admin can control the users and the application from the dashboard as you can see there are 5 users and the admin can see all the lands and jobs that are posting to verify either they are scam or real.

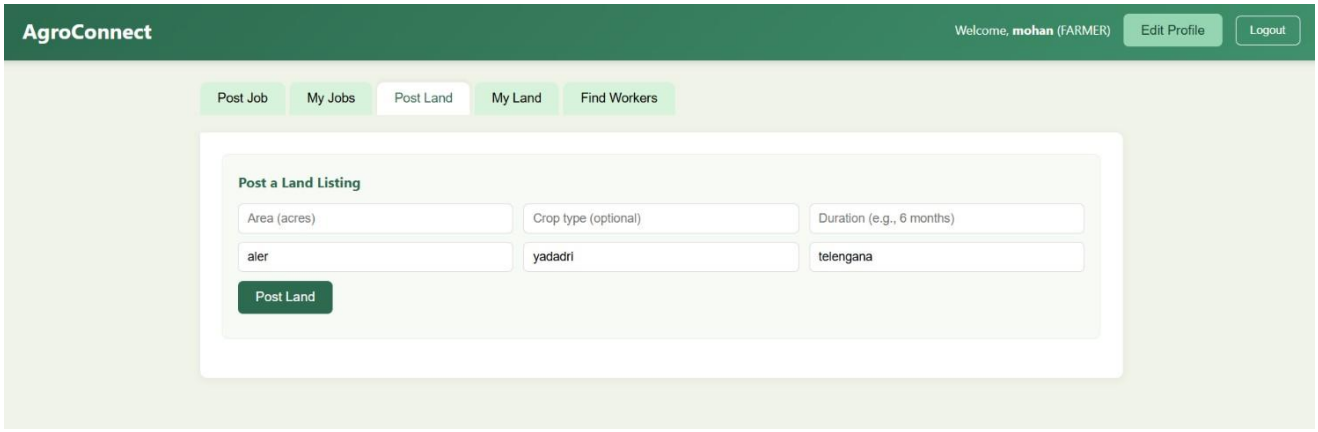


Fig. 8.4 LAND POSTING PAGE

This output screen shows the farmer's dashboard where he can post his land for kavulu to lend his land to them for farming

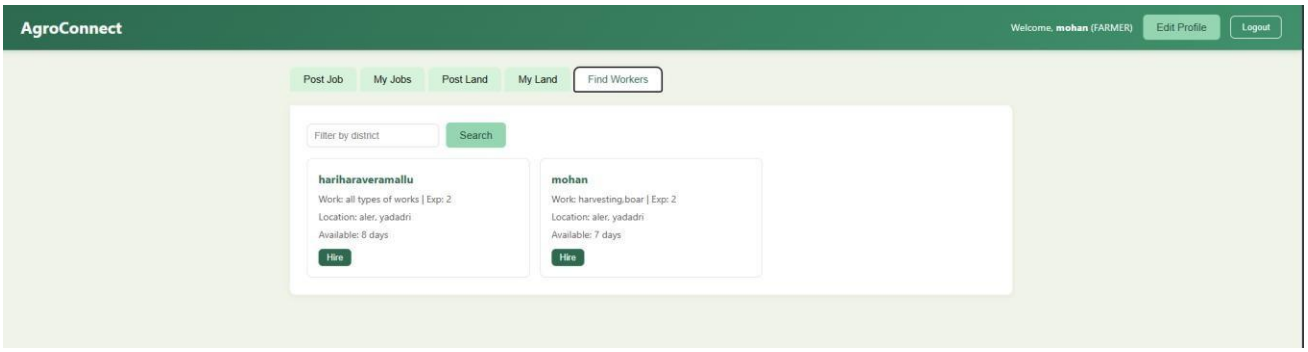


Fig. 8.5 WORKERS HIRING PAGE

The image describes where the farmer can hire the workers who want to work for farmer's and in this page the farmer can hire the nearby workers who are in this application.

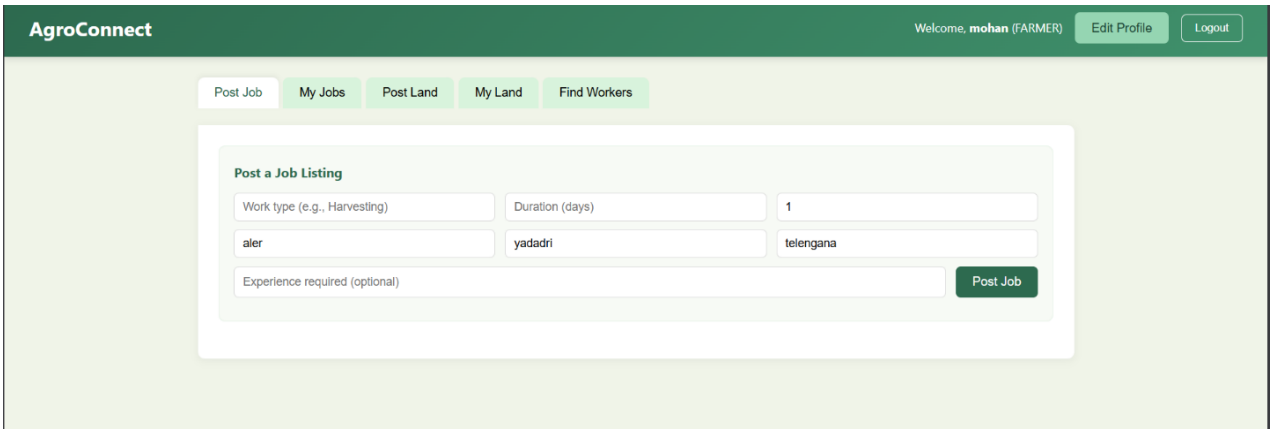


Fig. 8.6 JOBS POSTING PORTAL

This above image describes about the farmers dashboard where thay can post their jobs in the dashboard for the workers to get their jobs and how many days the farmer wants he can describe here.

CHAPTER-9

CONCLUSION

The AgroConnect platform manages to illustrate the combination of the latest practice of enterprise software and the logistical requirements of the Indian agricultural industry. The project has achieved this by closing the communication gap between the Farmers, Workers, and the "Kavulu" (tenant) farmers using a Java Spring Boot architecture and a MySQL relational database. The system substitutes the old and manual coordination process with a secure and role-based digital ecosystem that guarantees data integrity and real-time transparency. Technically speaking, the technical implementation confirms the efficiency of the Spring Data JPA framework in handling complex relational data, and JWT-based authentication prevents sensitive employment records to be accessed by unauthorized personnel. In addition to the code, AgroConnect addresses important market failures by enabling the rural labor force to circumvent exploitative intermediaries and enabling farmers to make their seasonal labor more economically efficient. In the end, this project is a holistic Agri-Tech solution that is capable of being deployed, which demonstrates that contemporary software engineering can be an effective driver of socio-economic change and productivity in the conventional industries.

CHAPTER-10

FUTURE ENHANCEMENTS

While the current version of AgroConnect successfully handles core workforce and land management, the following enhancements are proposed to evolve the platform into a comprehensive agricultural ecosystem:

10.1 Intelligent Workforce Optimization

- **AI-Driven Matchmaking:** Implementation of Machine Learning algorithms to suggest the "Best Match" for a farmer based on a worker's past ratings, skill sets, and geographic proximity.
- **Dynamic Wage Estimation:** An automated module to suggest fair daily wages by analyzing real-time market demand, crop complexity, and seasonal labor shortages.

10.2 Fintech & Secure Transactions

- **Digital Escrow Systems:** Integration of secure payment gateways (such as Razorpay or UPI) to hold wages in a secure escrow, releasing funds only upon verified job completion to build mutual trust.
- **Micro-Insurance Modules:** Collaborating with providers to offer "Per-Day" accidental insurance coverage for workers, activated automatically during their employment period via the app.

10.3 Precision Agriculture & IoT Integration

- **Smart Land Monitoring:** Linking land listings with IoT sensor data (Soil moisture, pH levels, and nitrogen content) to allow "Kavulu" farmers to assess land health remotely before requesting a lease.
- **Geofencing for Attendance:** Utilizing GPS-based geofencing to automatically verify worker attendance and "clock-in" status once they enter the designated farm perimeter.

10.4 Inclusive User Experience

- **Vernacular UI & Multilingual Support:** Expanding the interface to include regional languages (Telugu, Kannada, Tamil, etc.) to increase accessibility for rural populations.

- **Voice-to-Text Commands:** Implementing voice-driven navigation and data entry to assist users with lower digital literacy or those working in field conditions.

10.5 Agri-Sharing Economy

- **Equipment Rental Marketplace:** Extending the platform to allow farmers to list and rent out heavy machinery (Tractors, Harvesters, or Drones) to other farmers, optimizing resource utilization.
- **Supply Chain Integration:** Connecting farmers directly with local seed and fertilizer distributors to provide a "one-stop-shop" experience within the AgroConnect ecosystem.

CHAPTER-11

REFERENCES

1. Spring Framework Documentation: "Spring Boot Reference Guide," [Online]. Available: https://spring.io/projects/spring-boot. (Accessed: March 2026).
2. Jakarta EE Specifications: "Jakarta Persistence (JPA) 3.1 Specification for Object-Relational Mapping," [Online]. Available: https://jakarta.ee/specifications/persistence/3.1/.
3. MySQL Development Team: "MySQL 8.0 Reference Manual: Relational Database Management Systems," Oracle Corporation, [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/.
4. REST Architecture: Fielding, R., "Architectural Styles and the Design of Network-based Software Architectures," University of California, Irvine. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm.
5. AWS Cloud Services: "Amazon Relational Database Service (RDS) User Guide for MySQL," [Online]. Available: https://docs.aws.amazon.com/rds/.
6. Spring Security: "Spring Security Architecture and JWT Authentication Patterns," [Online]. Available: https://spring.io/projects/spring-security.
7. Agri-Tech Trends: "Digital Transformation in Indian Agriculture: Challenges and Opportunities," NITI Aayog Reports. [Online]. Available: https://www.niti.gov.in/sites/default/files/2021-02/Digital-Agriculture.pdf.
8. Hibernate ORM Documentation: "Hibernate User Guide: Object-Relational Mapping for Java," [Online]. Available: https://hibernate.org/orm/documentation/.